

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: D. Vijaya Sri

Roll No.: A24126552262

1. TITLE

Differentiate Between JavaScript Functions and Classes and Create Multiple Objects Using a Class

2. AIM

To understand the difference between JavaScript functions and classes and demonstrate how a class can be used to create multiple objects with common properties and methods.

3. OBJECTIVE

The objectives of this experiment are:

- To understand the purpose of JavaScript functions.
 - To understand classes, objects, properties, constructors, and methods.
 - To create multiple objects from a single JavaScript class.
 - To use DOM manipulation and an event listener to display object details.
 - To observe the common structure and different data of objects created from the same class.
-

4. THEORY

JavaScript Function: A function is a reusable block of JavaScript code designed to perform a particular task. It can accept parameters and return a value.

JavaScript Class: A class is a blueprint for creating objects. It defines common properties and methods that can be used by multiple objects.

Constructor: The constructor is a special method that runs when an object is created using the new keyword. It initializes the object's properties.

Object: An object is an instance of a class. Multiple objects can be created from one class, with each object containing its own property values.

In this experiment, a normal function named `greetStudent()` is used along with a `Student` class. Three `Student` objects are created using the same class. Each object has the common properties `name`, `course`, and `year`, and uses the common `getDetails()` method.

Basic flow: Browser → JavaScript Event → Class Objects → DOM Manipulation → Displayed Student Details

5. ALGORITHM / PROCEDURE

1. Create an HTML page containing a heading, description, button, and output area.
 2. Create a JavaScript function named `greetStudent()` to demonstrate a normal function.
 3. Define a `Student` class with a constructor.
 4. Initialize the student name, course, and year inside the constructor.
 5. Define the `getDetails()` method to return the student's details.
 6. Create three `Student` objects using the new keyword.
 7. Select the output element using DOM manipulation.
 8. Attach a click event listener to the button.
 9. When the button is clicked, display the greeting and all three `Student` objects.
 10. Open the HTML file in a browser and verify the output.
 11. Test the page before and after clicking the button.
-

6. PROGRAM

GitHub Folder Structure

```
Q5-JavaScript-Functions-and-Classes/  
├── src/  
│   ├── index.html  
│   ├── script.js  
│   └── style.css
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>JavaScript Functions and Classes</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h1>JavaScript Functions and Classes</h1>

    <p>
      This example demonstrates a normal function and a JavaScript class.
    </p>

    <button id="showStudentsBtn">Show Student Objects</button>

    <div id="output"></div>
  </div>

  <script src="script.js"></script>
</body>
</html>
```

script.js

```
// Normal JavaScript function
function greetStudent(name) {
    return "Hello, " + name + "!";
}

// JavaScript class
class Student {
    constructor(name, course, year) {
        this.name = name;
        this.course = course;
        this.year = year;
    }

    // Class method
    getDetails() {
        return `
            <div class="student">
                <h3>${this.name}</h3>
                <p><strong>Course:</strong> ${this.course}</p>
                <p><strong>Year:</strong> ${this.year}</p>
            </div>
        `;
    }
}

// Creating multiple objects from the same class
const student1 = new Student("Ravi", "CSE", "2nd Year");
const student2 = new Student("Anu", "CSE", "2nd Year");
const student3 = new Student("Kiran", "CSE", "3rd Year");

// DOM element
const output = document.getElementById("output");

// Event listener
document.getElementById("showStudentsBtn").addEventListener("click", function () {

    const greeting = greetStudent("Student");

    output.innerHTML = `
        <h2>${greeting}</h2>
        ${student1.getDetails()}
        ${student2.getDetails()}
        ${student3.getDetails()}
    `;
});
```

style.css

```
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  margin: 0;
  padding: 30px;
}

.container {
  max-width: 700px;
  margin: auto;
  background-color: white;
  padding: 25px;
  border-radius: 10px;
}

h1 {
  text-align: center;
}

button {
  padding: 10px 18px;
  cursor: pointer;
}

.student {
  margin-top: 15px;
  padding: 15px;
  border: 1px solid #ccc;
  border-radius: 8px;
}
```

7. CODE EXPLANATION

Code / Concept	Explanation
greetStudent()	A normal JavaScript function that returns a greeting message.
class Student	Defines a blueprint for creating Student objects.
constructor()	Initializes name, course, and year for each object.
this.name / this.course / this.year	Store property values belonging to the current object.
getDetails()	Class method that returns the formatted details of a Student object.
new Student(...)	Creates a new object from the Student class.
getElementById()	Selects the output element from the HTML document.
addEventListener()	Runs the specified code when the button is clicked.
innerHTML	Displays the generated student details inside the output element.

8. TEST CASES AND EXECUTION DATA

The experiment was tested using the actual browser execution shown in the submitted screenshots. Q5 does not accept user-entered form data, so invalid-input validation is not applicable.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Open index.html	Page loads with heading, description and button.	Page loaded with heading, description and button.	Pass
TC-02	Click Show Student Objects	Greeting and three Student objects should be displayed.	Greeting and Ravi, Anu and Kiran details were displayed.	Pass

Validation: The page was checked before clicking the button and after clicking it. The second state confirmed that the class objects and their common method were working in the browser.

9. OUTPUT

Output 1: Initial Webpage



Output 2: After Clicking Show Student Objects

JavaScript Functions and Classes

/FULL_STACK/01-observation-and-record/observation/Task%206/Q5-JavaScript-Functions-and-Classes/src/index.html

JavaScript Functions and Classes

This example demonstrates a normal function and a JavaScript class.

Show Student Objects

Hello, Student!

Ravi
Course: CSE
Year: 2nd Year

Anu
Course: CSE
Year: 2nd Year

Kiran
Course: CSE
Year: 3rd Year

10. RESULT

Thus, the JavaScript Functions and Classes experiment was successfully implemented. A normal JavaScript function was demonstrated, a Student class was created, and multiple Student objects were instantiated with common properties and methods. The application was executed in the browser and the displayed output verified the working implementation.

End of Observation Record